

gRPC Service Communication Setup in Kubernetes for OTA Platform

Objective

As a DevOps Engineer, my responsibility was to configure and enable **internal gRPC communication between Kubernetes pods** across multiple environments:

- Development (Dev)
- User Acceptance Testing (UAT)
- Production

Implementation Details

1. Kubernetes Service Discovery

I used **Kubernetes Service DNS** for internal communication between services. Each supplier service was exposed via a dedicated Kubernetes Service, allowing other pods to access it using a stable DNS name.

2. Environment-Based Configuration

Separate namespaces and service names were configured for each environment to ensure isolation and flexibility:

- **Development Namespace:** `travilo`
- **Production Namespace:** `production`

Each supplier service has a unique DNS entry and port configuration.

Example gRPC Service Endpoints

Kubernetes provides built-in DNS-based service discovery, allowing pods to communicate with each other using predictable and consistent DNS names instead of hardcoded IP addresses.

Development:

<http://travilo-us-bangla-air-service.travilo.svc.cluster.local:50056>

UAT / Production:

<http://travilo-us-bangla-air-service.production.svc.cluster.local:50056>

Explanation of gRPC endpoints:

- **Service Name:** travilo-us-bangla-air-service
- **Namespace:** travilo
- **Service Type Indicator:** svc
- **Cluster Domain:** cluster.local

The DNS resolves the service and the port (50056) directs traffic to the gRPC server running in the container.

3. Service & Deployment Configuration

- Modified **Kubernetes Service YAML** files to expose the required gRPC ports for each supplier.
- Updated **Deployment YAML** files to ensure containers listen on the correct gRPC ports.
- Ensured consistent naming conventions across environments for easier management and scaling.

```
apiVersion: v1
kind: Service
metadata:
  name: #{Deployment}#
spec:
  selector:
    app: #{Deployment}#
  ports:
    - name: http
      port: 80
      targetPort: 80
    - name: grpc
      port: 50053
      targetPort: 50053
```

4. Application Configuration

The development team configured the gRPC endpoints within the application's `appsettings` files. Since they manage service-to-service communication logic, they used the DNS names provided to integrate gRPC clients accordingly.

Key Responsibilities

- Designed and implemented internal DNS-based service discovery
- Configured Kubernetes Services for gRPC communication
- Managed environment-specific service naming and namespaces
- Exposed and standardized gRPC ports across all supplier services
- Collaborated with developers to ensure seamless integration